

Region Resonances

Methods Primer

A Guide to the Statistical and Network Methods

Jure Skarabot · May 2026

Companion documents: Summary · Technical Appendix · Data Appendix · Region Reference — Identity

Introduction

The Region Resonances tool answers a simple question: given a free-form text query, which of the 59 wine regions in the Soul of Wine corpus does the query most resemble in cultural identity? The procedure relies on a small number of techniques drawn from natural language processing and high-dimensional geometry: text embeddings, the unit sphere, cosine similarity, nearest-neighbour search, and a calibrated display transformation.

This primer explains each of these techniques in the order they appear in the matching pipeline. The treatment assumes comfort with elementary algebra and basic geometry but no prior exposure to embeddings or vector-space retrieval. Full mathematical treatment is in the companion *Technical Appendix*.

1. Embeddings: From Text to Numbers

What an embedding is. An embedding is a function that converts a piece of text — a word, a phrase, a paragraph — into a list of numbers called a vector. The function is produced by training a large language model on enormous quantities of text; the result is that texts with similar meanings are assigned similar vectors, where similarity is defined precisely by the geometry of the space the vectors live in.

Dimension. The number of entries in each vector is called the dimension of the embedding. Region Resonances uses OpenAI’s `text-embedding-3-small` model, which produces 1536-dimensional vectors. Every region passage and every user query becomes a list of 1536 real numbers.

Why this is useful. The embedding map encodes meaning into geometry. Once two pieces of text are represented as vectors, the question “how similar are they in meaning?” becomes the question “how close are these two vectors?” — a question that can be answered with arithmetic. The matching procedure exploits this directly: the user’s query and the 59 region passages are all embedded into the same 1536-dimensional space, and the regions whose vectors are closest to the query vector are returned as the matches.

What the model has learned, and what it has not. Modern embedding models are trained on text from books, articles, web pages, and other sources, learning patterns of co-occurrence: which words appear in similar contexts, which phrases tend to be used together, which sentences

tend to convey related ideas. The result is a remarkably accurate sense of semantic similarity for general English. The model does not, however, have a special understanding of wine. Its judgment about which two passages are similar in meaning reflects the patterns of general English, not expert wine knowledge. This is sufficient for the cultural identity narratives at hand, which are written in plain language; it would be insufficient for highly technical viticultural text.

2. The Unit Sphere

Normalisation. After producing each vector, the embedding model rescales it so that its overall magnitude is exactly one. Mathematically, this means dividing each vector by its own length. The result is a vector that points in the same direction as the original but always has length 1.

What it means geometrically. A vector of length 1 in a d -dimensional space lies on the surface of a sphere of radius 1 centred at the origin. The collection of all such vectors is called the unit sphere. After embedding, every region and every query is represented as a point on the surface of the same unit sphere in 1536 dimensions. Two regions whose narratives describe similar cultural character are nearby points on this sphere; two regions with very different cultural character are far-apart points.

Why the sphere matters. The matching procedure does not need to consider the magnitude of vectors at all — only their directions. Two passages can be of very different lengths, with very different raw word counts, but the embedding model strips out this magnitude information and leaves only the direction. The unit-sphere geometry is what enforces this: the matching is about where the vector points, not how long it is.

3. Cosine Similarity

What cosine similarity measures. For two vectors on the unit sphere, the cosine of the angle between them is a natural measure of how aligned they are. If two vectors point in exactly the same direction, the angle between them is zero and the cosine of zero is one. If they are perpendicular, the angle is ninety degrees and the cosine is zero. If they point in exactly opposite directions, the angle is one hundred eighty degrees and the cosine is minus one.

Cosine similarity in formula. For two vectors $\mathbf{x}$ and $\mathbf{y}$, cosine similarity is the inner product divided by the product of the two lengths. When both vectors lie on the unit sphere, the lengths are both one, the division has no effect, and the cosine similarity reduces to the inner product alone:

$$\cos(\mathbf{x}, \mathbf{y}) = x_1y_1 + x_2y_2 + \cdots + x_dy_d.$$

Computing the cosine similarity between a query and one region is therefore $d = 1536$ multiplications followed by 1535 additions — arithmetic that takes microseconds.

Where Region Resonances uses it. Every comparison the tool makes is a cosine similarity. The user submits a query; the tool computes the cosine similarity between the query vector and each of the 59 region vectors; the regions are sorted in descending order of similarity; the top five are returned. This is the entire matching procedure.

4. Nearest-Neighbour Search

What it does. Once each comparison reduces to a single number (the cosine similarity), finding the best match is straightforward: compute the 59 similarities and pick the largest. This is called nearest-neighbour search, because the chosen region is the nearest neighbour of the query in the geometric sense.

Top- k extension. Region Resonances returns not just the single nearest neighbour but the five nearest. This is called top- k retrieval with $k = 5$. The motivation is that semantic similarity is rarely sharply peaked: a query like *old soul* resonates with several regions to varying degrees, and the user benefits from seeing the small set of competing matches rather than a single answer presented as if it were definitive.

Computational cost. With only 59 regions in the corpus, the nearest-neighbour search is trivial: 59 cosine similarities and a sort. For larger corpora, specialised data structures (k-d trees, inverted indices, approximate nearest-neighbour libraries) become necessary. None of that complication applies here.

5. The Display Transformation

Raw cosine values are misleading. A natural impulse would be to display the raw cosine similarity to the user as a percentage: a similarity of 0.62 would be shown as 62%. This would be misleading. Although cosine similarity is mathematically bounded between -1 and $+1$, the values actually observed for the Region Resonances corpus occupy a much narrower range, roughly 0.15 to 0.65. A cosine of 0.62 is in fact a strong match, almost as good as the strongest semantically meaningful match the system ever returns; presenting it as “62%” implies failure.

Why the range is narrow. The narrowness is a generic property of high-dimensional embeddings of natural-language text. Almost any two passages of English share a substantial baseline of common semantic structure — they are both grammatical English, share vocabulary, refer to a shared world. The embedding model encodes this shared structure into a common region of the sphere, and the signal that distinguishes one passage from another lives in a relatively narrow range above the baseline. Numbers below 0.15 occur only for pathological inputs (random byte sequences, unrelated languages); numbers above 0.70 occur only for queries that are themselves paraphrases of corpus text.

Clip-and-rescale. To present scores that use the full visual scale, Region Resonances applies a piecewise-linear transformation. Two calibration constants $s_{\min} = 0.15$ and $s_{\max} = 0.70$ define the empirical operating range. The raw cosine s is mapped to a display value $\tilde{s}$ in the interval $[0, 100]$ via

$$\tilde{s} = \text{clip}\left(\frac{s-0.15}{0.70-0.15}, 0, 1\right) \times 100,$$

where clip pins values below 0 to 0 and values above 1 to 1. A raw cosine of 0.15 displays as 0; a raw cosine of 0.70 displays as 100; a raw cosine of 0.42 displays as 49.

What this transformation preserves. The transformation has three properties that matter for interpretation: it is bounded between 0 and 100 (so displayed scores fit on a percentage scale), it is monotonically increasing on the operating range (so the displayed ranking always matches the underlying cosine ranking), and it is affine on the operating range (so equal cosine differences map to equal display differences). Two regions whose displayed scores differ by 10 are equally distinguishable in the underlying space, whether the scores are 40 and 50 or 80 and 90.

Why not softmax. A common alternative for converting raw similarity scores to a percentage is the softmax transformation, which turns the 59 similarities into a probability distribution that sums to 100%. Softmax also preserves the ranking, but it has two drawbacks for this application. It depends on a temperature parameter that has no principled value, and different choices produce dramatically different displayed numbers for the same underlying ranking. And it is multiplicative rather than additive in the score gap, so a small cosine difference between near-tied results becomes a large displayed difference, exaggerating the apparent confidence of the top match. Clip-and-rescale, by contrast, treats equal cosine differences as equal display differences — the honest representation of what the matching procedure has found.

6. The Pipeline in Summary

The full Region Resonances pipeline can be read as a sequence of five stages:

1. *Corpus assembly.* The 59 region passages are assembled by concatenating each region’s one-word identity metaphor with its full Layer 1 identity narrative.
2. *Corpus embedding (offline).* Each passage is converted to a 1536-dimensional unit-norm vector by the OpenAI `text-embedding-3-small` model. The 59 vectors are stored in a static file shipped with the tool.
3. *Query embedding (online).* The user’s query is sent to the same embedding model, which returns a single 1536-dimensional unit-norm query vector.
4. *Nearest-neighbour search.* The cosine similarity between the query vector and each of the 59 region vectors is computed in one matrix–vector product; the regions are sorted in descending order of similarity; the top five are selected.
5. *Display.* The raw cosine values are mapped to a $[0, 100]$ display score by clip-and-rescale and presented to the user along with the matching regions.

What the procedure guarantees is that two queries with very similar meaning produce very similar rankings, and that two regions with very similar narratives are returned together for many queries. What the procedure does not guarantee is that the rankings agree with any particular human’s intuition — that property is inherited from the embedding model, and is an empirical question rather than a mathematical one.

Further Reading

For a general introduction to vector-space retrieval: Manning, C. D., Raghavan, P. & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. Chapter 6.

For sentence and passage embeddings specifically: Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proc. EMNLP-IJCNLP 2019*, 3982–3992.

For the geometry of high-dimensional vectors: Aggarwal, C. C., Hinneburg, A. & Keim, D. A. (2001). On the surprising behavior of distance metrics in high dimensional space. *Proc. ICDT 2001*.

For the embedding model used: OpenAI (2024). *New embedding models and API updates*. OpenAI Platform documentation.